

OptArrow Structural Architecture Document

1. System Context

OptArrow is an advanced integration engine designed to facilitate high-performance connections between optimization clients and solver backends. It achieves this by supporting a diverse range of optimization scenarios through a robust transport architecture centered on Python and Julia backends. Stakeholders, including Software Engineers, Researchers, Data Scientists, and Optimization Specialists, utilize OptArrow for solving mathematical optimization problems, specifically linear programming (LP) and quadratic programming (QP). The system processes input problems from various environments, such as MATLAB, and executes them via the most suitable backend solver, delivering results in the language of the original problem specification.

2. Components & Responsibilities

Core Components:

- **Python and Julia Backends:** These serve as the primary computational engines, ensuring cross-environment compatibility and high-performance problem-solving.
- **Apache Arrow:** Handles in-memory transport for efficient data exchange between different language components.
- **Client Interfaces:** Lightweight interfaces for external environments like MATLAB, enabling seamless bidirectional communication with OptArrow's core architecture.

The division of responsibilities ensures each component serves a precise function, such as routing, problem interpretation, and computational execution, optimizing the system's overall performance.

3. Interfaces & Data Flow

OptArrow's architecture leverages Apache Arrow for in-memory data transport, focusing on cross-language interoperability essential for operations involving Python and Julia.

- **Interface Mechanics:** Problems are ingested as Python dictionaries or equivalent Julia structures. Apache Arrow transforms these into a standardized in-memory Inter-Process Communication (IPC) format, allowing them to be processed by the desired solver backend without losing data fidelity.
- **Data Flow Dynamics:** Upon IPC transformation, data is routed to either the Python or Julia backend, depending on the contextual solver requirements. Post-computation, the results are converted back into the original language format for output, ensuring consistency with the input environment.

4. Runtime Interactions & Sequence

Execution Overview:

1. **Problem Identification:** The problem source is determined—Python, Julia, or an external interface such as MATLAB.
2. **Data Transformation:** Input data is formatted into Arrow IPC for internal processing.
3. **Solver Routing:** The problem is directed to a high-performance solver determined by specified criteria, which may include HiGHS due to its open-source advantages.
4. **Computation and Output:** The solver processes the input, and results are returned in the originating language format for immediate application by stakeholders.

This sequence ensures that data integrity and execution efficiency are maintained throughout the runtime process.

5. Technology Decisions & Rationale

Decision Criteria:

- **Apache Arrow:** Chosen for its robust in-memory data transfer and cross-language compatibility, essential for the heterogeneous operating environment of OptArrow.
- **HiGHS Solver:** Adopted due to its free, open-source nature and popularity within the scientific and developer communities.

The choice of technologies aligns with the need for a streamlined architecture supporting OptArrow's high-performance objectives.

6. Deployment & Constraints

Current deployments of OptArrow emphasize seamless operation across diverse programming environments, ensuring neither Python nor Julia versions create constraints for users. However, deployment is limited by the maturity of existing architecture, suggesting room for expansion in stable governance processes.

7. Known Architectural Gaps

Several gaps exist that necessitate targeted improvements:

- **Governance & Documentation:** There is an absence of formal architecture governance and structured documentation processes, affecting consistency and strategic development.
- **Traceability:** The lack of formal system traceability undermines potential alignment between user feedback, decisions, and implemented changes.
- **Alternative Protocols:** The current reliance on Apache Arrow was chosen based on intended architectural goals rather than empirical trials, suggesting a need for exploring alternative data-transfer protocols as OptArrow scales.

Future efforts are expected to address these gaps, involving the implementation of industry-standard architectural frameworks and decision documentation such as Architecture Decision Records (ADRs), aligning more closely with ISO 33061 standards.